Runtime Toolsets

What it does and how it works at runtime · plugin 0.14.0 · Unreal Engine 5.8 · 2026-10-27

Runtime Toolsets lets an AI agent build and run a game **while the game is running**. The plugin turns the live Unreal world into **32 toolsets of 335 tools** that an agent calls with plain JSON: build terrain and buildings, light the scene, place effects and UI, add physics, AI and game rules, then check the result with screenshots. Nothing needs the Unreal Editor: the same tools work in a packaged game, including one streamed from the cloud.

1. What it does

An agent works the way a person would at a live editor, but through tools. A typical session:

- **Look:** session.list_toolsets and skills.get_skills tell the agent what exists and how to use it; capture.screenshot shows the result.
- **Build:** build.create_terrain, build.create_primitive / create_assembly (any shape, real materials), gis plots, PCG scatter of trees, splines for roads and rivers; the Python kit adds whole buildings (27 types: apartment to petrol station) and districts.
- **Dress:** lighting moods and time of day, art styles (realistic, toon, lowpoly, noir, retro, stylized), materials, Niagara effects, HUD text and world labels.
- **Make it playable:** character.create or logic.spawn_player, logic rules (when touch, add score), AI agents on a navmesh, Gameplay Ability System effects, StateTrees, physics.
- **Change and undo:** every change is a scene-graph node or command, so history.undo, checkpoints and rollback work across all toolsets.

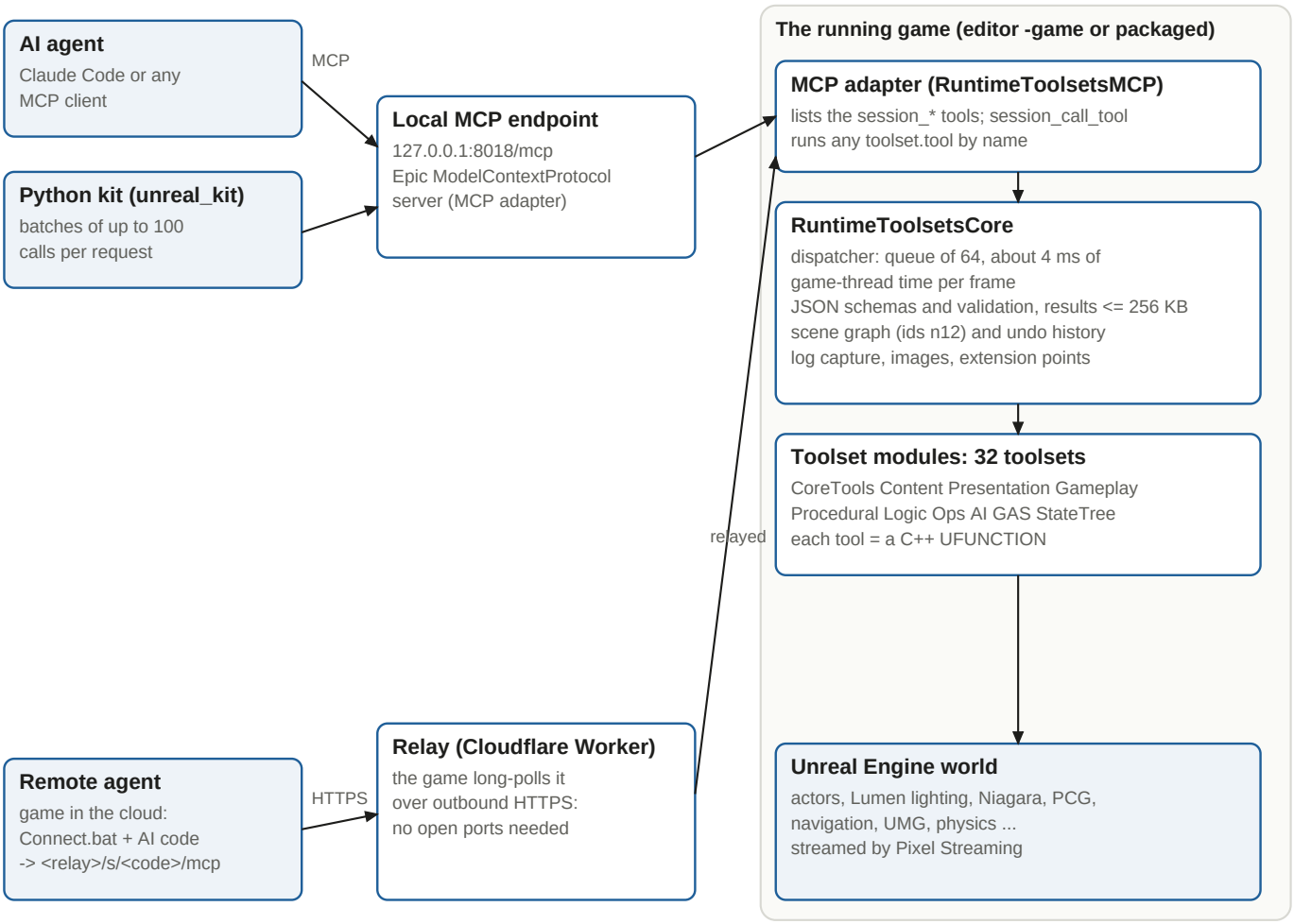
Toolsets (read from the plugin; module = the code module that registers it):

Toolset	Tools	Module	What it covers
actors	9	CoreTools	Actors: spawn, find, move, attach, hide, inspect and destroy actors in the running world.
ai	18	AI	Real Unreal AI at runtime: navigation meshes, AI-controlled agents that walk on them, AI perception (sight and hearing), noise events, behavior trees and blackboards.
build	17	Content	Builder: terrain with biomes, islands and rivers; primitives and generators (arches, roofs, steps, lattices); assemblies of many parts; profile solids; real meshes and GPU instances; ...
camera	10	Presentation	Player view control.
capture	3	CoreTools	Capture: images of the running game for visual checks.
character	3	Logic	Generic playable character: any game's hero built from settings instead of a genre preset.
console	6	CoreTools	Console: console commands, console variables (cvars), scalability and user settings of the running game.
game	5	Gameplay	The running game: map, game mode, players, pause and time scale; move players or actors around.
gas	20	GAS	Gameplay Ability System at runtime: inspect and drive attributes, gameplay effects, abilities, gameplay events, loose tags and cues on any actor, including builder objects with no ...
gis	13	Procedural	GIS: land plots (parcels) of any polygon shape.
history	8	Core	Undo/redo, transactions, checkpoints and rollback (per session)
input	4	Gameplay	Simulated player input for the local player: raw keys/buttons/axes and Enhanced Input actions and mapping contexts.
layout	9	Content	Layout: arrange many objects at once by their bounds (children included): align, distribute, snap to a grid, drop to the ground, rows / grids / circles, seeded jitter, and finding free ...
level	8	Gameplay	Levels and content of the running game: streaming levels, level instances, World Partition data layers, map travel and a search over the cooked asset registry.
lighting	10	Presentation	Outdoor lighting at runtime: sun (directional light), sky light, sky atmosphere, volumetric clouds, height fog and the global post process (exposure, grading).

Toolset	Tools	Module	What it covers
log	3	CoreTools	Log and stats: read the game log, subscribe to log categories as pushed events, and read frame/memory stats.
logic	43	Logic	Game logic without code: turns a built scene into a playable game with variables, event rules (when click / key / button / timer / touch / start ...
materials	8	Presentation	Materials at runtime: material slots and parameters on live meshes, and material parameter collections (world-wide values).
ops	10	Ops	Orchestration and testing: run many tool calls in one request (optionally atomic, one undo step), wait for game state, and run automation tests inside the running game (also packaged ...
pcg	23	Procedural	Procedural Content Generation: build and run PCG graphs (the same node-by-node workflow as Epic's editor PCG toolset) and scatter builder objects along or inside splines.
physics	8	Gameplay	Physics and collision queries in the running world: traces, overlaps, simulation state, pushes and ragdolls.
profiling	6	Ops	Performance profiling of the running game: frame timing percentiles, hitches, memory, GPU info and Unreal Insights traces.
reflection	11	CoreTools	Reflection: inspect and change any object's properties and call its Blueprint-callable functions, like Epic's object tools.
scene	21	CoreTools	Scene graph: the hierarchy of nodes (actors and groups) created by the tools.
session	8	Core	Health, build info, toolset discovery and call_tool
skills	4	CoreTools	Skills: authoritative guidance for kinds of work (outdoor lighting, art styles, PCG graphs, building, game logic).
spline	2	Procedural	Splines: smooth or straight paths through points, optionally shown as a swept mesh (roads, rivers, walls, pipes).
statetree	7	StateTree	StateTree at runtime: list and inspect compiled StateTree assets, run them on any actor (a StateTreeComponent is added when needed), send events and read the active states.
style	4	Presentation	Art style: one call restyles the whole scene (realistic, stylized, lowpoly, toon, retro, noir, ...).
tags	7	AI	Gameplay Tags at runtime: browse the tag table (Epic: list_tags, get_tag_info), give any actor tags, and query them.
ui	17	Presentation	Live UMG at runtime: inspect, create and change on-screen widgets, HUD text without assets, world labels, and the activity panel.
vfx	10	Presentation	Niagara effects at runtime.

2. How it fits together

Three ways in, one engine inside. Agents speak the Model Context Protocol (MCP) to the game, either directly on this machine or through a small relay when the game runs somewhere nothing can connect to (a cloud GPU streaming the game). Every request ends in the same dispatcher on the game thread.



Way in	Address	When
Local MCP	http://127.0.0.1:8018/mcp (port: -RuntimeToolsetsPort=N)	Agent and game on one machine; editor -game or packaged.
Relay	https://<relay>/s/<AI code>/mcp	Game in the cloud: the game long-polls the relay over outbound HTTPS and shows a short AI code; Connect.bat links the agent once. Nothing is installed.
Python kit	unreal_kit.zip next to the game (uk.connect())	Scripts that build much at once: batches of up to 100 calls per request; reconnects by itself when the game reloads a level.

3. How a tool call runs at runtime

From the agent's request to the change on screen, in order:

1. **Request.** The agent calls an MCP tool. In the default Meta mode the game lists only the session_* tools; the agent discovers toolsets with session.describe_toolset and runs any tool through session_call_tool {name: "build.create_primitive", arguments: {...}}. This keeps the agent's tool list short.
2. **Queue.** The call joins the dispatcher's queue (at most 64 waiting). All engine work happens on the game thread, so the dispatcher drains the queue in slices of about 4 ms per frame: the game keeps rendering and streaming at full rate while an agent builds.
3. **Validate.** Arguments are checked against the tool's JSON schema (generated from the C++ UFUNCTION signature and its doc comment). A wrong type or a missing argument comes back as an error with a hint, and nothing changes.

4. **Run.** The tool runs as a command: it spawns or edits actors, components and materials through the normal engine APIs. Changes that last (spawn, move, material, light) are recorded in the session's history with how to undo them; new objects become scene-graph nodes with stable ids (n12) and paths (house/door).
5. **Answer.** The result is JSON (at most 256 KB, or the call fails with a hint to narrow it). Capture tools attach an image. Slow tools (navigation build, tests, waits) finish asynchronously, within 30 s by default.
6. **See it.** The world updates in the next frame; the player, or a Pixel Streaming viewer, sees it at once.

4. The scene graph, undo and safety

- **Nodes.** Everything the tools create is a node: an actor plus its source recipe (prim:, mesh:, inst:, vfx: ...). From the recipe a node can be rebuilt: undo/redo, duplicate, templates (scene.save_template) and export/import of whole trees.
- **Soft delete.** Deleting hides the actor and keeps it, so undo restores it exactly; scene.clear_all clears the session's nodes in one undoable step.
- **History.** history.undo / redo, transactions (several calls = one step), named checkpoints and rollback; 200 steps per session by default.
- **Guard rails (Config/DefaultGame.ini).** Reflection may only touch allowed object roots and never denied classes or functions; console commands can be deny- or allow-listed; level actors the tools did not make cannot be destroyed unless allowed; toolsets and single tools can be switched off.

5. Building it into a game

- **Plugins.** RuntimeToolsets (the toolsets, no external content needed) and RuntimeToolsetsMCP (the MCP adapter and relay client). Both load in Development builds; Shipping and Test builds leave them out.
- **Content.** The core plugin carries its own materials (/RuntimeToolsets). Project assets it can use, such as scanned surfaces or a rippled-sand texture, are named in the project config as asset slots (AssetSlots in DefaultGame.ini), so the plugin itself depends on nothing.
- **Build and test.** Build the editor target, then run the automation tests (RuntimeToolsets.*: 223 tests, including schema snapshots that flag any change to a tool's public surface).
- **Package.** A normal BuildCookRun (Win64 Development, Pixel Streaming on). The relay URL and token are written into the config only while packaging and never committed. The Python kit and its examples are copied next to the exe.

6. Extending it

A new toolset is a C++ class deriving from URuntimeToolsetDefinition whose static UFUNCTIONs carry meta=(AICallable): the doc comment becomes the tool's description, the parameters its JSON schema, and the module registers the class at startup. Extension plugins add more without touching the core:

- **Recipes:** RegisterRecipes adds ready-made scenes to ops.list_recipes / ops.run_recipe.
- **Asset slots:** SetAsset points the core at an extension's materials (weather-aware ground, scanned surfaces) or a project's third-person character.
- **Lighting owner:** RegisterLightingOwner tells the lighting tools that a sky system drives the light, so they refuse with a pointer to its tools instead of fighting it.
- **Skills:** any URuntimeAgentSkill subclass is found by itself and served by skills.get_skills.
- Toolsets for bought content packs (FluidNinja fluids, Ultra Dynamic Sky, Road Creator Pro roads) were built this way in a separate plugin; they are kept out of this project for now.

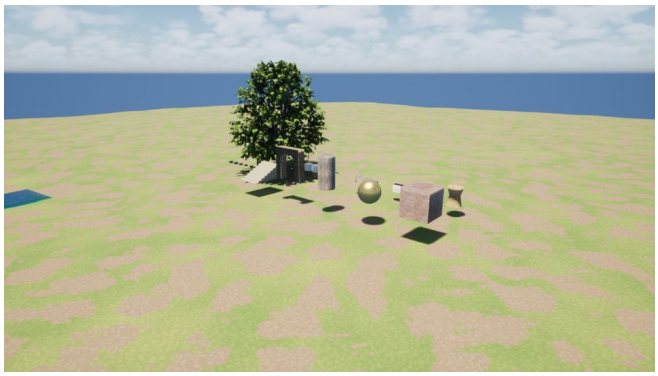
7. Known limits

- Development and editor builds only: the plugins are excluded from Shipping and Test configurations.

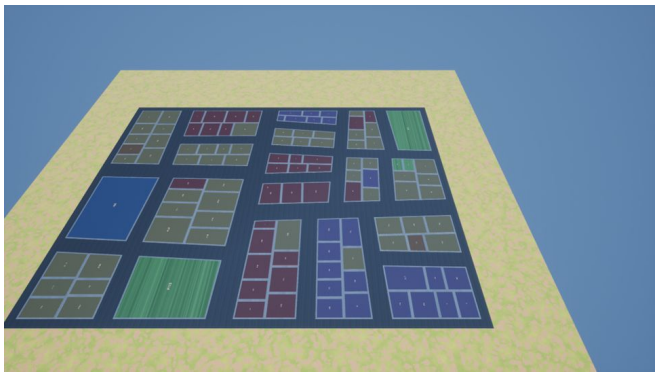
- session.call_tool cannot call itself; the Python kit already runs every call through it.
- Results over 256 KB fail: scene.get_node on a very large building (hundreds of parts) does; use scene.list_tree with fields ["bounds"].
- Screen UI (HUD text, logic HUDs) is not a scene node, so scene.clear_all leaves it on screen.
- The retro style keeps its pixel look without colour posterising (that step flattened the image).

8. Proof

Every toolset was exercised in a running game (447 of 448 calls passed; the one refusal is the expected self-call) and all 27 building types were built in the packaged game. Screenshots and the saved responses are in **Docs/Proof**; Scripts/proof regenerates them. A few of them:



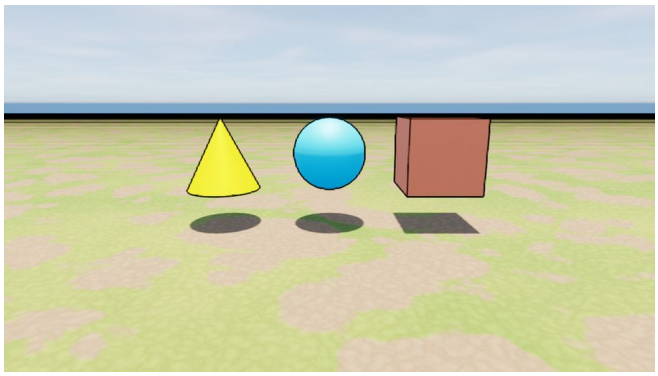
build: terrain, primitives, assembly, mesh, river



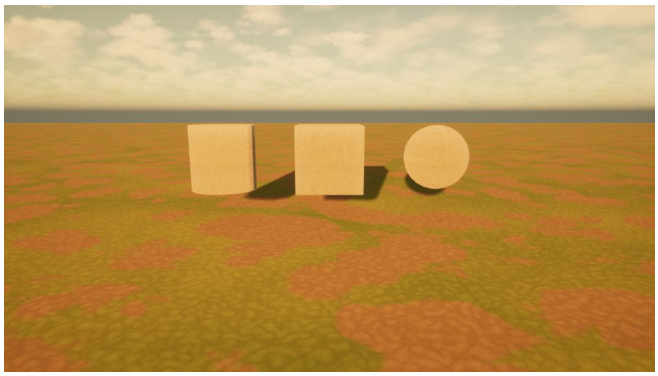
gis: generated plots and streets



pcg: a forest scattered inside a spline



style: toon



lighting: sunset mood



kit: office tower